

☆☆ NIVEL INTERMEDIO ☆☆

Testing con IA — Estrategia y Prompts

Ejercicio Individual — Tipos de pruebas, Frameworks y Diseño de Prompts

Módulo 3 · Contesta con lo que vimos en clase. Puedes consultar tus apuntes.

Campo	Rellena aquí
Nombre del alumno/a	ANGEL ECHENIQUE
Fecha	24/03/2026

□ **Instrucciones:** Responde con lo que vimos en clase. No es necesario escribir código en las partes teóricas. En la parte práctica, usa VS Code y pega el resultado en el espacio indicado.

PARTE 1 — ¿Por qué usar IA para testing? — Clasifica y justifica

En clase vimos **6 ventajas clave** de usar IA para generar tests. Para cada una, escribe si aplica principalmente a **U** (Unitarias), **I** (Integración) o **A** (Ambas), y explica con un ejemplo concreto:

Ventaja de la IA	U / I / A	Ejemplo concreto de cuándo la usarías
Genera tests rápidamente a partir del nombre de una función o su comportamiento esperado.	A	Para generar test para funciones y endpoints. Con esto aceleraríamos el desarrollo y permitiríamos validar funcionalidades desde el inicio.
Sugiere casos edge: entradas vacías, valores extremos, tipos inesperados o errores de usuario.	A	Para detectar inputs vacíos, límites o errores en cualquier nivel. Con esto mejoramos la robustez del software.
Crea mocks y stubs automáticamente para simular servicios externos, bases de datos o APIs.	A	Para simular APIs en unitarias o servicios externos en integración y saber si mi flujo de sistema esta funcionando bien.
Aumenta la cobertura cubriendo caminos positivos, negativos, excepciones y errores.	A	Para detectar fallos antes de que llegue a producción.

Reduce errores humanos: evita omisiones, tests mal estructurados o duplicaciones.	A	Como asistente técnico para que revise el código y me proponga mejoras.
Enseña buenas prácticas como la estructura AAA	A	Para generar tests estructurados y legibles.
(Arrange-Act-Assert) y nombres descriptivos.	A	Para todos los tests, especialmente en pruebas unitarias y de integración.

PARTE 2 — Unitaria vs. Integración — Clasifica y compara

Para cada escenario indica si se necesita una prueba **U** (Unitaria) o **I** (Integración). Luego justifica brevemente tu decisión:

U / I	Escenario a probar	¿Por qué es de ese tipo?
U	Una función que calcula el descuento de un producto según el tipo de cliente.	Podemos probarla aisladamente al ser una función simple.
I	El flujo completo: un usuario se registra, hace login y accede a su perfil.	Simulamos el recorrido de un usuario o proceso completo.
U	Un método que valida si una contraseña tiene mayúsculas, números y más de 8 caracteres.	Porque valida lógica interna sin depender de otros componentes.
I	El endpoint POST /orders que crea un pedido, lo guarda en base de datos y envía un email.	Tenemos que verificar que múltiples componentes trabajen junto correctamente.
U	La clase CarritoCompra con sus métodos añadir_producto() y calcular_total().	Para testear métodos internos, propiedades, constructores y comportamiento encapsulado.
I	La interacción entre el módulo de autenticación y el módulo de permisos de usuario.	Verifica que los módulos se comuniquen correctamente (servicios, controladores, modelo).
U	Una función pura que convierte una lista de temperaturas de Celsius a Fahrenheit.	Podemos probarla aisladamente al ser una función simple.
I	El módulo de pagos que llama a la API de Stripe, registra la transacción y notifica al usuario.	Porque involucra API externa, base de datos y notificaciones.

2.2 — Completa la tabla comparativa

Rellena las celdas vacías basándote en lo que vimos en clase:

Característica	Prueba Unitaria	Prueba de Integración
¿Qué prueba?	Una función o clase aislada	Verifican que múltiples componentes trabajen juntos correctamente.
Velocidad	≤ Milisegundos	Más lenta (Segundo)
¿Usa mocks?	Si, frecuente.	A veces, las que usemos en ese proyecto.
Dependencias	Ninguna (aislada).	Reales (BD, API, etc.).
Número de tests	Muchos.	Pocos (decenas) .
Cuándo se ejecuta	En cada commit.	En CI/CD o en fases posteriores.
¿Qué fallo indica?	Error en lógica interna alguna función o validación	Interacción rota entre módulos .

PARTE 3 — Frameworks de testing — Une, clasifica y razona

Completa la tabla escribiendo el **lenguaje principal** de cada framework y un **caso de uso real** en el que lo usarías:

Framework	Lenguaje principal	Caso de uso real — ¿Cuándo lo usarías?
pytest	Python	Para testing moderno en proyectos Python (APIs con FastAPI, scripts, backend).
Jest	JavaScript	Testing de frontend (React) y backend Node.js. Ej: validar componentes UI o endpoints.
JUnit 5	Java	Testing en aplicaciones Java (Spring Boot). Ej: servicios backend empresariales.
Vitest	JavaScript	Testing rápido en proyectos modernos con Vite (React, Vue).
RSpec	Ruby	Testing en aplicaciones Ruby on Rails. Ej: validación de modelos y controladores.

PHPUnit	PHP	Testing en aplicaciones PHP (Laravel, Symfony). Ej: lógica backend y controladores.
xUnit	.NET (C#)	Testing en aplicaciones .NET. Ej: APIs en ASP.NET Core.
testify	Go	Testing en Go con utilidades extra (asserts, mocks). Ej: servicios backend en Go.

3.2 — pytest vs unittest: Verdadero, Falso... y por qué

Marca **V** (Verdadero) o **F** (Falso) y escribe una justificación breve. Sin justificación **no puntúa**:

V / F	Afirmación	Justificación
F	pytest necesita más código (boilerplate) que unittest porque obliga a usar clases.	pytest usa menos boilerplate y no requiere clases, se pueden usar funciones simples.
V	Con <code>@pytest.mark.parametrize</code> puedes probar múltiples valores de entrada en un solo test.	Permite ejecutar el mismo test con diferentes inputs de forma eficiente.
F	Si heredas un proyecto en unittest no puedes ejecutarlo con pytest sin reescribirlo todo.	pytest es compatible con unittest y puede ejecutar esos tests sin cambios.
V	pytest-cov mide qué porcentaje del código está cubierto por los tests.	Genera métricas de cobertura indicando qué partes del código se ejecutan.
F	Para proyectos de IA/ML como PyTorch o FastAPI, unittest es el estándar de la industria.	pytest es el estándar más usado por su simplicidad y potencia.
V	La IA puede generar tests para ambos frameworks, pero conoce mejor pytest.	La mayoría de ejemplos y datasets están basados en pytest, por eso genera mejores resultados.

PARTE 4 — Tipos de casos de prueba — Diseña los tuyos

Para la función `calcular_descuento(precio, porcentaje)` diseña al menos **3 casos de cada tipo** y explica qué validas:

Recordatorio: **Normal (N)** = casos donde todo funciona (happy path). **Límite (L)** = valores extremos o frontera. **Error (E)** = entradas inválidas que deben provocar un error controlado.

Tipo(N,L,E)	Entrada (precio, porcentaje)	Resultado esperado	¿Qué estás validando?
N	(100, 10)	90	Cálculo correcto de descuento básico (caso típico)
N	(200, 25)	150	Aplicación correcta de porcentaje mayor
N	(50, 5)	47.5	Correcto funcionamiento con valores pequeños
L	(0, 10)	0	Límite inferior del precio (no debe fallar)
L	(100, 0)	100	Límite inferior del porcentaje (sin descuento)
L	(100, 100)	0	Límite superior del porcentaje (descuento total)
E	(-100, 10)	Error / False	Precio negativo no válido
E	(100, -5)	Error / False	Porcentaje negativo no permitido
E	(100, 150)	Error / False	Porcentaje mayor a 100 inválido

PARTE 5 — Diseña el prompt profesional para generar tests

⚠ **Situación:** Tienes esta función Python en un sistema de gestión de empleados:

```
def calcular_salario_netto(salario_bruto: float, horas_extra: int, categoria: str) -> float:
    """Calcula el salario neto aplicando IRPF según categoría
    (junior/senior/manager),
    añadiendo el pago de horas extra (15€/hora) y validando que los valores
    sean positivos."""
```

5.1 — Identifica los 6 elementos del prompt

Rellena la tabla indicando qué escribirías en cada elemento del prompt para esta función concreta:

Elemento del prompt	Qué escribirías exactamente para esta función
1. Función o módulo a testear	Función <code>calcular_salario_netto(salario_bruto: float, horas_extra: int, categoria: str)</code> que calcula el salario neto según categoría (junior, senior, manager), añade horas extra (15€/hora) y valida valores positivos.
2. Framework deseado	Pytest.
3. Casos edge a incluir	Salario = 0, horas_extra = 0, salario negativo, horas negativas, categoría inválida, valores muy altos.
4. Mocks necesarios	No necesarios (función pura sin dependencias externas).
5. Formato de salida	Tests con <code>@pytest.mark.parametrize</code> , incluyendo inputs y resultado esperado.
6. Explicación de cada test	Breve descripción indicando qué regla o caso se está validando.

5.2 — Escribe el prompt completo

Escribe el prompt completo y profesional que enviarías a la IA. Incluye todos los elementos y al menos **3 casos edge específicos**:

Quiero que actúas como un ingeniero de QA experto en testing automatizado con pytest.
Necesito que generes tests para la siguiente función: def calcular_salario_neto(salario_bruto: float, horas_extra: int, categoria: str) -> float: Calcula el salario neto aplicando IRPF según categoría (junior/senior/manager), añadiendo el pago de horas extra (15€/hora) y validando que los valores sean positivos.
Contexto: Esta función pertenece a un sistema de gestión de empleados donde es crítico calcular correctamente los salarios.
Requisitos a cubrir: - Casos normales (cálculo correcto para cada categoría). - Casos límite: salario 0, horas extra 0 - Casos error: salario negativo, horas negativas, categoría inválida - Casos con valores altos para comprobar estabilidad.
Formato de salida: - Usar pytest. - Usar @pytest.mark.parametrize. - Incluir entrada y resultado esperado. - Añadir una breve explicación por cada caso.
Restricciones: - No usar librerías externas adicionales. - Mantener estructura clara (AAA). - Generar código limpio y legible.

5.3 — ¿Qué harías después de recibir los tests?

Completa la tabla con lo que harías tras recibir el código generado por la IA:

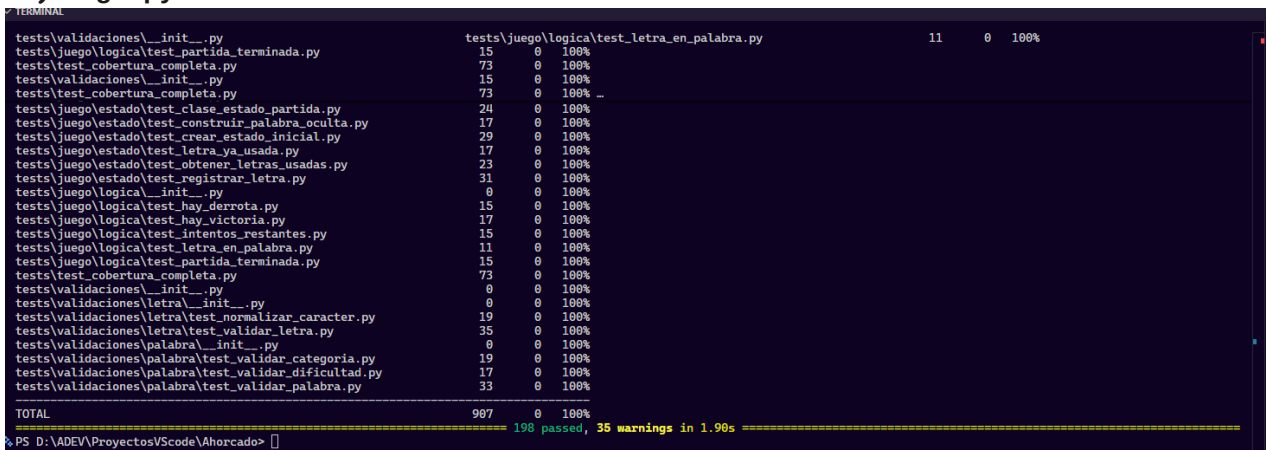
Acción que realizarías	¿Por qué es importante hacerlo?
Revisar cada test generado uno por uno	Para detectar errores lógicos, falsos positivos o tests mal diseñados.
Comprobar que los mocks son correctos	Para asegurar que simulan correctamente el comportamiento real (aunque aquí no haya mocks).
Añadir casos edge que la IA no incluyó	La IA suele olvidar casos extremos o poco comunes porque no tiene el criterio humano.
Ejecutar pytest y verificar que todos pasan en verde	Para validar que los tests funcionan correctamente en ejecución real.
Documentar qué tests fueron generados por IA	Para trazabilidad, mantenimiento y auditoría futura.

PARTE 6 — Retos opcionales — Para nota extra

Completa alguno de estos retos si terminas antes de tiempo:

Reto	Descripción
🔍 Plugin pytest-cov	Investiga qué hace pytest-cov y ejecuta pytest --cov=tu_modulo en VS Code. Haz una captura del informe de cobertura y explica qué significa el porcentaje que aparece.
⚡ Tests parametrizados	Escribe un test con @pytest.mark.parametrize para calcular_descuento que pruebe al menos 5 combinaciones de precio y porcentaje. Explica qué ventaja tiene frente a 5 tests separados.
🔍 Contexto de negocio	Reescribe el prompt de la Parte 5 añadiendo contexto financiero real. Compara qué tests adicionales genera la IA con el contexto frente a los que generó sin él.

1=) Plugin pytest-cov



```

tests\validaciones\__init__.py          tests\juego\logica\test_letra_en_palabra.py    11    0  100%
tests\juego\logica\test_partida_terminada.py  15    0  100%
tests\test_cobertura_completa.py          73    0  100%
tests\validaciones\__init__.py           15    0  100%
tests\test_cobertura_completa.py         73    0  100%
tests\juego\estado\test_clase_estado_partida.py  24    0  100%
tests\juego\estado\test_construir_palabra_oculta.py  17    0  100%
tests\juego\estado\test_crear_estado_inicial.py  29    0  100%
tests\juego\estado\test_letra_ya_usada.py  17    0  100%
tests\juego\estado\test_obtener_letras_usadas.py  23    0  100%
tests\juego\estado\test_registrar_letra.py  31    0  100%
tests\juego\logica\__init__.py            0    0  100%
tests\juego\logica\test_hay_derrota.py     15    0  100%
tests\juego\logica\test_hay_victoria.py    17    0  100%
tests\juego\logica\test_intentos_restantes.py  15    0  100%
tests\juego\logica\test_letra_en_palabra.py  11    0  100%
tests\juego\logica\test_partida_terminada.py  15    0  100%
tests\test_cobertura_completa.py          73    0  100%
tests\validaciones\__init__.py            0    0  100%
tests\validaciones\letra\__init__.py       0    0  100%
tests\validaciones\letra\test_normalizar_caracter.py  19    0  100%
tests\validaciones\letra\test_validar_letra.py  35    0  100%
tests\validaciones\palabra\__init__.py     0    0  100%
tests\validaciones\palabra\test_validar_categoria.py  19    0  100%
tests\validaciones\palabra\test_validar_dificultad.py  17    0  100%
tests\validaciones\palabra\test_validar_palabra.py  33    0  100%
-----
TOTAL                                     987    0  100%
===== 198 passed, 35 warnings in 1.98s =====
PS D:\ADEV\ProyectosVScode\Ahorcado>

```

EXPLICACIÓN DE LAS COLUMNAS

En el informe aparecen estas columnas:

Columna	Significado
Stmts	Número de líneas de código ejecutables
Miss	Líneas que NO han sido ejecutadas
Cover	Porcentaje de cobertura

EJEMPLO REAL DEL INFORME

mostrar_victoria.py 26 0 100%

Interpretación:

- 26 líneas de código
- 0 líneas sin ejecutar
- 100% cubierto por tests

Significa que **todas las rutas de ese archivo han sido probadas**

RESULTADO GLOBAL

TOTAL 907 0 100%

Interpretación:

- 907 líneas de código en total
- 0 líneas sin ejecutar
- **100% de cobertura**

DETALLE IMPORTANTE DEL OUTPUT

198 passed, 35 warnings

Interpretación:

- Todos los tests han pasado
- Existen **35 warnings** (problemas menores)

Ejemplo de warnings:

- Conexiones a base de datos no cerradas
- Posibles fugas de recursos

CONCLUSIÓN FINAL: El porcentaje de cobertura indica qué parte del código ha sido ejecutada por los tests. En este caso, el 100% significa que todas las líneas han sido cubiertas, pero esto no garantiza que el sistema esté libre de errores, ya que pueden existir fallos lógicos o advertencias como las detectadas en el informe.

2=)Tests parametrizados

```
import pytest

@pytest.mark.parametrize("precio, porcentaje, esperado", [
    (100, 10, 90),
    (200, 25, 150),
    (50, 5, 47.5),
    (100, 0, 100),
    (100, 100, 0),
])

def test_calcular_descuento(precio, porcentaje, esperado):
    assert calcular_descuento(precio, porcentaje) == esperado
```

Ventaja frente a 5 tests separados

Un solo test:

- Más limpio
- Menos código duplicado
- Más fácil de mantener
- Escalable (añades casos fácilmente)

Parametrize = 1 test → muchos casos

3=) SIN CONTEXTO

Actúa como un ingeniero de QA especializado en testing automatizado en Python.

Genera tests para la siguiente función:

```
def calcular_salario_neto(salario_bruto: float, horas_extra: int, categoria: str) -> float:
```

```
    """Calcula el salario neto aplicando IRPF según categoría (junior/senior/manager),  
    añadiendo el pago de horas extra (15€/hora) y validando que los valores sean positivos."""
```

Requisitos:

- Generar casos válidos (cálculo correcto del salario)
- Generar casos inválidos (valores negativos, categoría incorrecta)
- Incluir casos límite (salario 0, horas extra 0, valores altos)

Formato de salida:

- Usar pytest
- Usar @pytest.mark.parametrize
- Incluir parámetros de entrada y resultado esperado

Restricciones:

- No usar librerías externas adicionales
- Mantener el código limpio y legible

3=)CON CONTEXTO

Actúa como un QA especializado en sistemas financieros.

Genera tests para la función calcular_salario_net.

Contexto:

Esta función se usa en un sistema real de gestión de nóminas. Es crítico evitar errores ya que afecta a pagos reales a empleados y cumplimiento legal.

Incluye:

- Casos por categoría (junior, senior, manager)
- Validación de impuestos (IRPF)
- Casos límite (salarios altos, cero)
- Casos error (datos negativos, categorías inválidas)
- Casos realistas (empleados con horas extra frecuentes)

Formato:

pytest con parametrize y explicación por test.

Aspecto	Prompt SIN contexto	Prompt CON contexto financiero
Enfoque	Técnico y genérico	Orientado a negocio real.
Tipo de tests	Básicos (V, I, L)	Más completos y realistas.
Profundidad	Baja-media	Alta.
Casos críticos	Puede omitirlos	Los incluye (riesgo financiero).
Cobertura	Limitada	Más amplia.
Lógica de negocio	No considerada	Sí (IRPF, categorías).
Escenarios reales	No	Sí (empleados, horas extra, salarios reales).